Engineering reliable agentic AI systems

Session 1. System Architecture, the foundation.

Saturday 29 August 2026. 10:00 Central.

Rick Hightower. Spillwave. Packt workshop.

Engineers who already live in the tools.

- Software, AI, platform, DevOps, architects, tech leads.
- You already use Claude Code, Codex, Cursor, Gemini, or similar.
- The failure is not that you cannot prompt. It is that prompting does not scale.
- Instructor: Rick Hightower. Principal architect for agentic AI at Spillwave.

Four artifacts. You leave with all four.

01	02	03	04
A running autonomous loop, in the first hour	A reusable evaluation harness	One live research assistant over MCP	A production architecture you can hand to your team

Four hours of teaching. Module 2 does not get cut.

BLOCK	START	MINUTES
Open	10:00	10
Module 1 anatomy + lab	10:10	45
Break	10:55	15
Module 2 harness	11:10	55
Break	12:20	15
Module 3 research + MCP	12:35	40
Break	13:30	15
Module 4 production	13:45	35
Close	14:20	10

Prompting dies under volume.

- One clever prompt works once.
- Ten tickets a day, it drifts.
- A hundred, and nobody remembers what good looked like.
- The bottleneck is not the model. It is you, reading every diff.

The unit of work is a controlled loop, not a single generation.

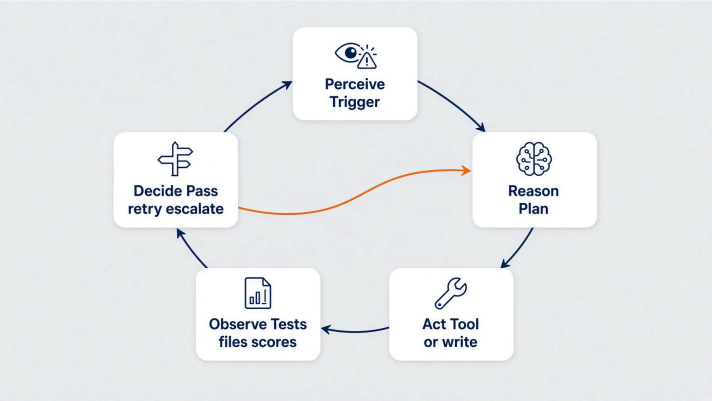
You write the system that triggers, acts inside a scope, verifies against a contract, remembers on disk, and stops or escalates under explicit exits.

Prompting does not scale under volume. The leverage moves to the design of the loop itself. A generator with a while loop is not this.

A loop is not "call the model until it says done."

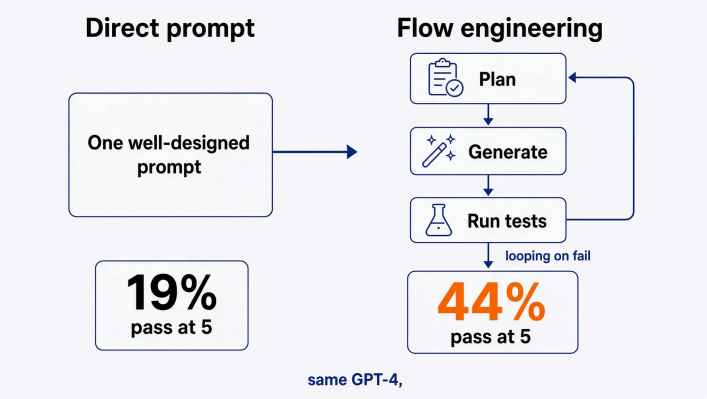
A production loop is a state machine. Every iteration:

The primitive cycle is still ReAct. The product is the outer control system.



Yao et al., ReAct, arXiv:2210.03629

The jump is the flow, not the prompt.



AlphaCodium, CodeContests validation set. Same model. The flow did that.

Ridnik, Kredo, Friedman. arXiv:2401.08500

Flow engineering is more calls, and still cheaper than you in the loop.

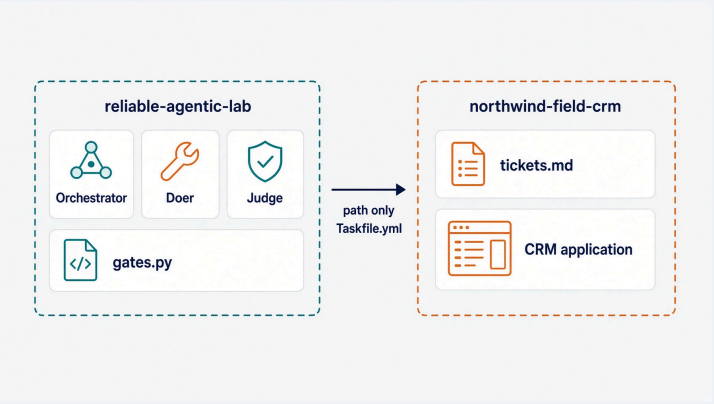
AlphaCodium uses about 15 to 20 LLM calls per solution. A pass@5 submission is roughly 100 calls.

The lesson for this room is not the exact number. It is that iteration against tests beat a clever prompt on the same weights.

The object is a CRM, and it lives in another repo.

- Northwind Field CRM. Customers. Sales tasks.
- Not a ticketing app. Too meta.
- The engine never imports it. You point the loop at a path.
- First ticket: add a due date. Vague on purpose.

Monday morning, you point this at your backlog.



The interface is `Taskfile.yml` plus `junit.xml`. That is the only contract the loops need.

The clock, and permission to fall behind.

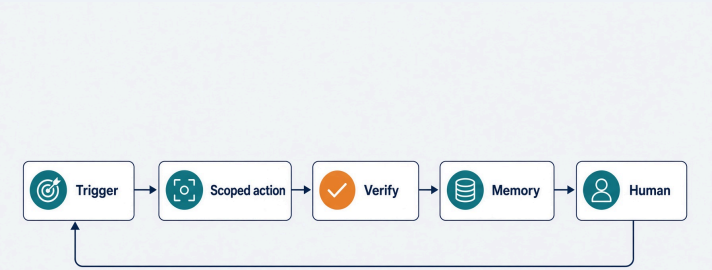
- 10 minutes open. 45 this module. Then a break.
- The lab is 25 minutes of typing, not 45.
- Stuck? Stop typing and watch.
- Copy `solutions/sol1_enhancer/.claude/` and you continue with a working artifact.

Nobody leaves this room behind.

Anatomy of an agent loop

Trigger. Action. Verify. Memory. Human oversight.

Five parts. Verify is the one that is not optional.

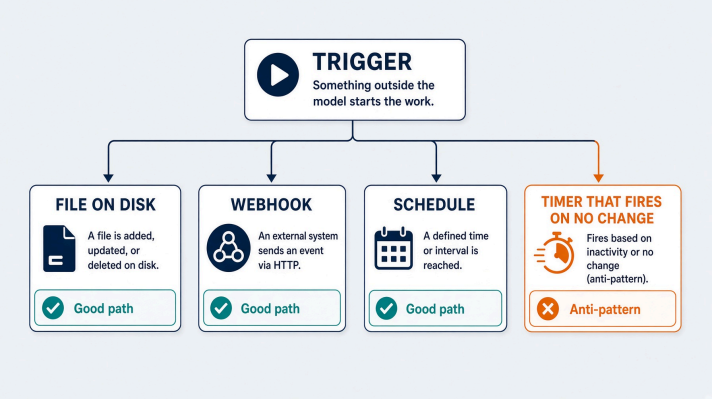


Five parts. **Verify** is the one that separates a loop from a script that calls a model.

Trigger

- Something outside the model starts the work.
- Ours today: a draft markdown ticket in the target repo.
- Not a chat. Not "hey, add due dates."
- In production it is a webhook or a schedule, and it fires only when the branch head actually moved.

The shape of a trigger changes. The rule does not.

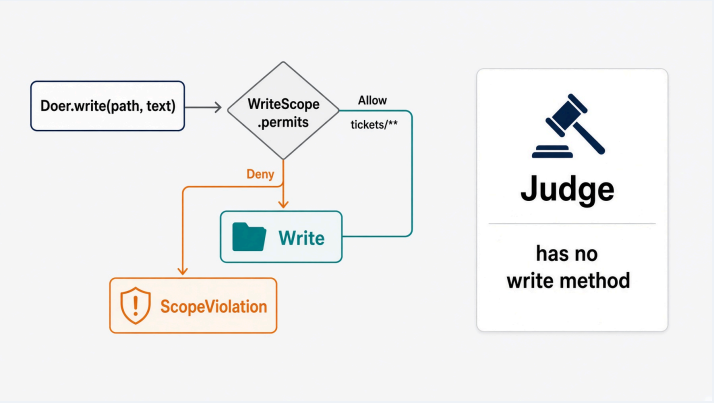


Action, inside a scope you declared

- A **doer** writes files. Only inside a declared scope.
- The scope lives in `.loop.yml`, in the target repo.
- It is enforced at the tool boundary, not in the prompt.

An agent can argue its way past an instruction. It cannot argue its way past a tool it was never given.

Write scope is a type, not a polite request.



Deny always beats allow. In Python, `Judge` has no `write` method. In the Claude Code lab, `enhancer-judge` and `enhancer-doer` carry no write tool in their agent definitions.

never the same process.

Verify

- A **judge** scores the result. It reports, and that is all it does.
- The judge holds no write path. Not a rule. A missing method.
- Today it answers: is this ticket a contract a test could fail?

If verify is "looks good to me," you do not have a loop. You have a generator.

Never let the AI verify its own done.

Same model plus same context plus same reasoning process is not a checker. The split is architectural.

Judge has no write method to call.

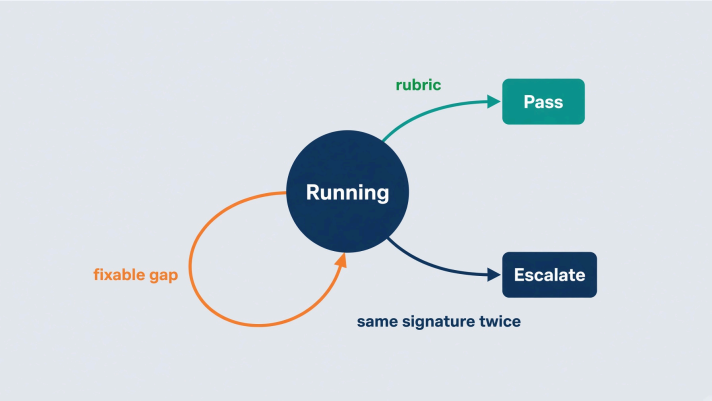
```python
@dataclass
class Judge(Role):
    """Scores work. Holds no write path.

    There is deliberately no `write` method on this class.
    Adding one is not a convenience. It is the end of the split.
    """

    def read(self, relative: str) -> str:
        return (self.repo / relative).read_text(encoding="utf-8")
```

`loops/roles.py`

Three exits, and no fourth. Python holds the loop.



`pass`, `retry`, `escalate`. The one people miss is stable failure.

The same gaps twice is not progress. Stopping is the feature.

`signature` is what failed, not how it was worded. Two equal signatures mean the last attempt changed nothing.

`loops/gates.py` · `decide()`

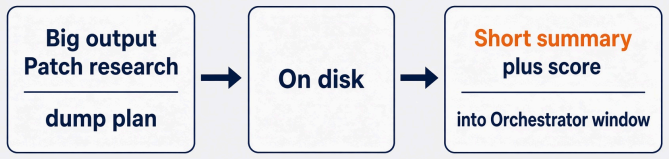
Memory, and why the context window is not it

Liu et al. 2024 measured it. Accuracy is highest when the fact sits at the **start** or the **end** of the context. Move it to the middle and accuracy drops by more than 30%.

- Reproduced on GPT-4, Claude, MPT-30B, and Cohere Command.
- So state goes on disk: the ticket, the trace, the plan.

Liu et al., TACL 2024. arXiv:2307.03172

Big output goes to a file. Only a short summary returns.



Big output goes to a file

That rule is why the planner is its own subagent in Module 2.

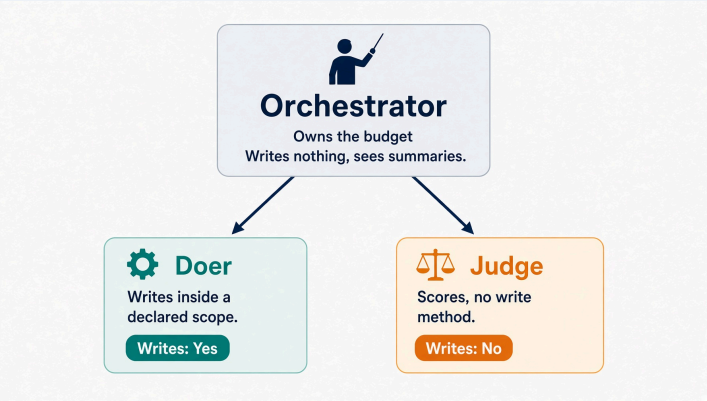
Human oversight

- The loop proposes. A human accepts.
- Today, the loop rewrites the ticket. You decide it is a contract.
- The only human input on the issue is a comment: `LGTM`.
- In Module 2 the loop opens a pull request. It still does not merge.

Oversight is a designed step, not a hope.

Merge, money, and production deploy stay human.

Three parts. The object is the only variable.



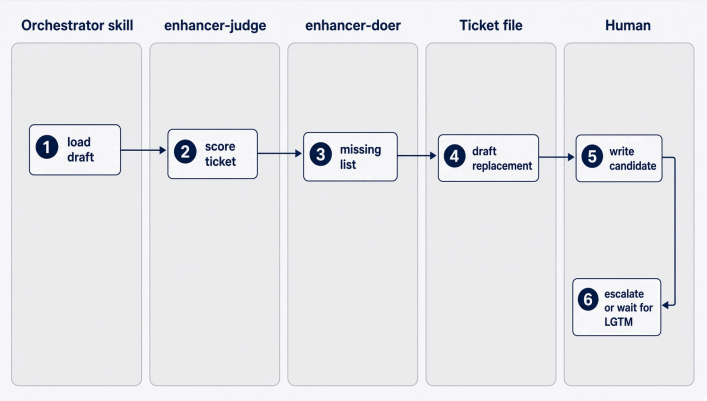
Same graph, four objects. Learn it once.

MODULE	OBJECT	LAB
1	A draft ticket	Enhancer
2	A ready ticket, and the code that satisfies it	Implementer
3	A question	Research
4	A failing pull request	Fixer

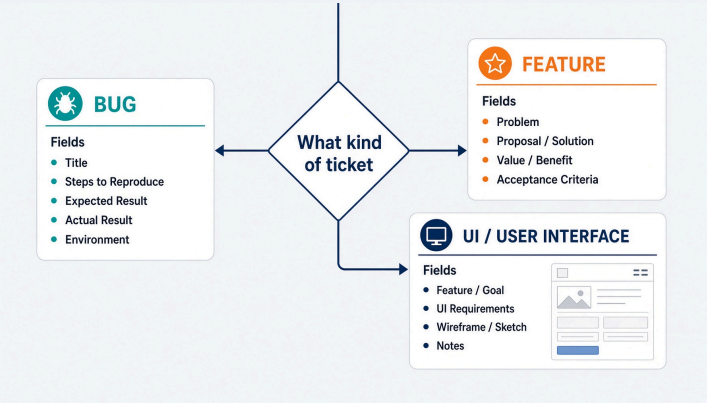
Context windows reset. Ticket files do not.

Read `.harness/last-enhancer.json` in the lab. That is the trace.

Ticket enhancer. Vague in, contract out.



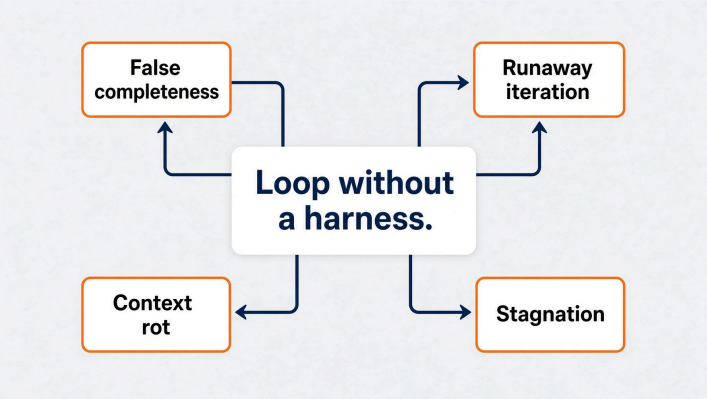
A feature ticket is a contract a test can fail.



Deterministic where it can be. A criterion that names a section is checkable without a model.

`loops/criteria.py`

If you take away verify, stop, scope, or disk, the loop collapses.

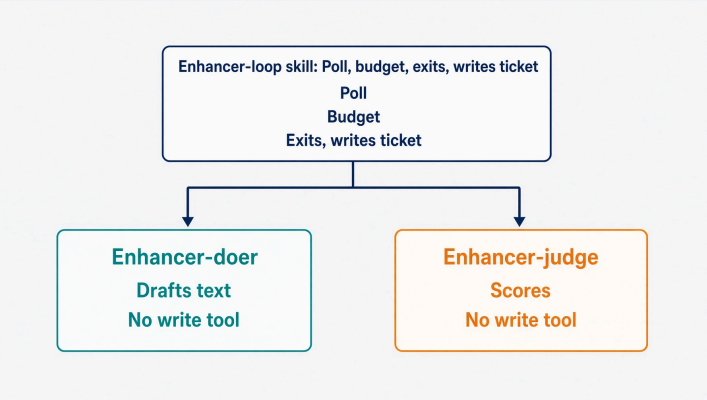


FAILURE	ROOT CAUSE	SYMPTOM
False completeness	Self-verification	"Done" with placeholders
Runaway iteration	Missing hard stop	Token burn
Context rot	Unmanaged history	Quality falls after a handful of turns
Stagnation	No progress detection	Same two states forever

Lab 1. The Ticket Enhancer

25 minutes. A vague ticket in. A ready contract out.

A skill that owns the loop. Two agents with no write tools.



The orchestrator is the only writer. That is a real limitation of the skill form: nothing enforces the round budget except the skill following its own instructions.

Lab 1. The Ticket Enhancer. 25 minutes.

```
cd labs/lab1_enhancer
# Claude Code plugin. Not a Python stub.

task run --
# First poll needs no comment. LGTM is the only human action.
```

Build the plugin from `prompts/claude-code.md`.
Codex, Grok, and OpenCode have native answers.

This lab writes no application code, so the push gate stays quiet.

Falling behind is fine: copy `solutions/sol1_enhancer/.claude/`.

Read the trace. The interesting run is the one that stops.

```
round 1: feature, not ready
  missing: why it is worth doing
  missing: acceptance criteria a test can fail
round 2: feature, not ready
  missing: why it is worth doing
  missing: acceptance criteria a test can fail

gate: escalate
reason: the same rows failed twice. The loop is not converging.
```

An iteration that burns tokens and reproduces the identical failure is not progress. Stopping is the feature.

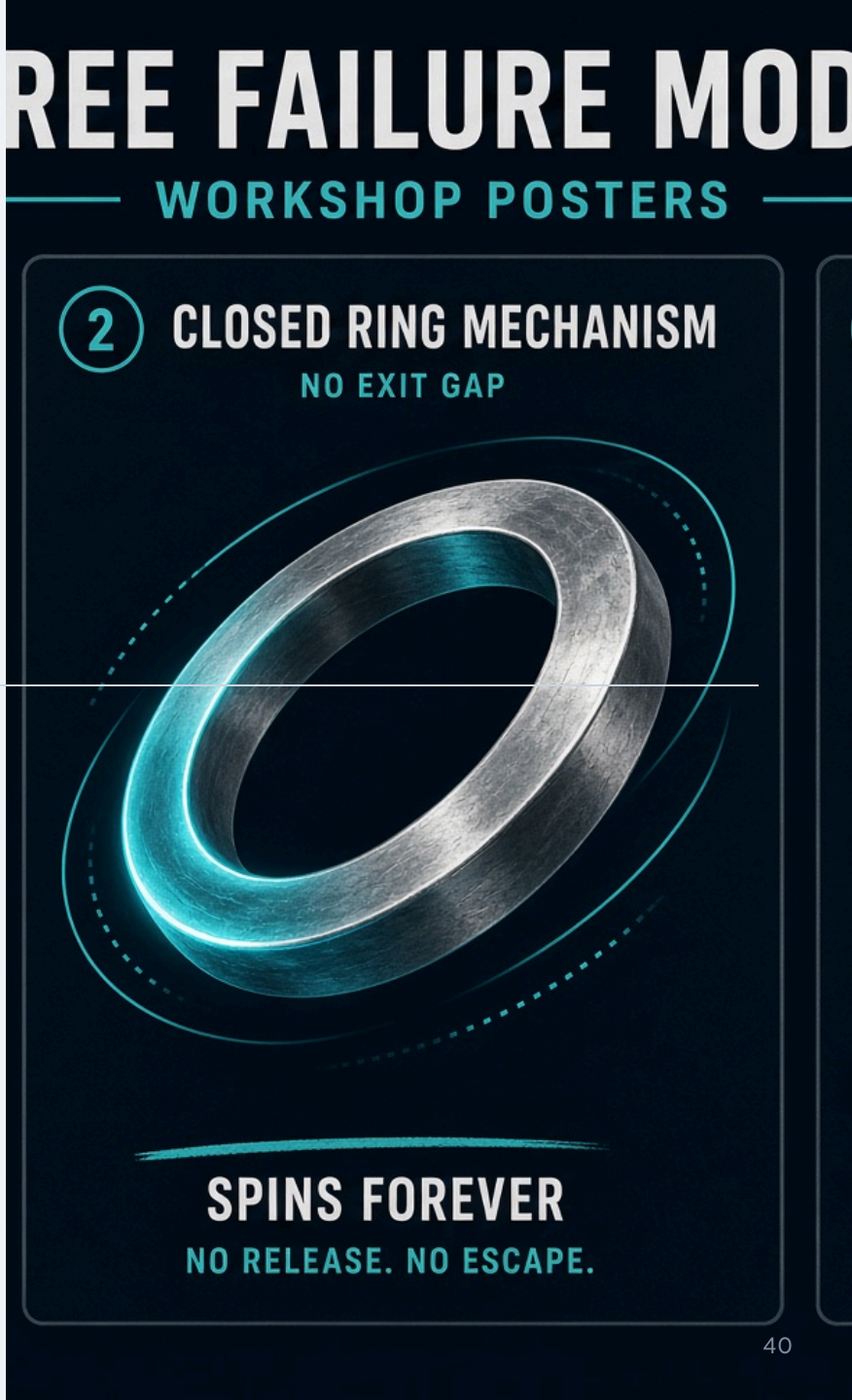
Where this breaks at scale

Module 1 is a loop that can run once. Module 2 makes it honest.

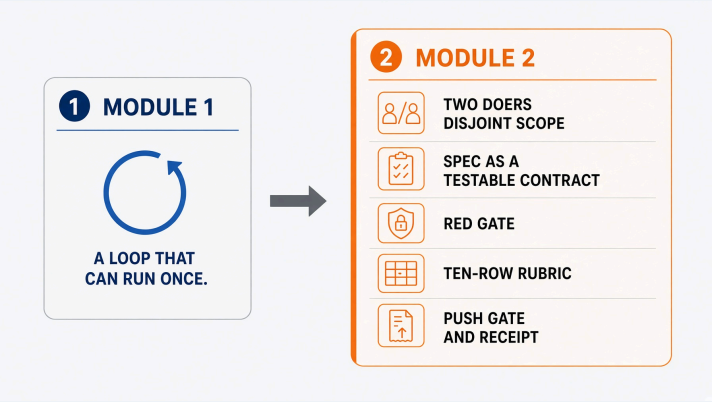
Where this breaks at scale

- **No contract.** The doer invents the field type, and you find out in review.
- **No stop.** It edits forever, and the bill arrives on Monday.
- **No scope.** It weakens the test instead of fixing the code.
- **Context rot.** The whole repo goes into the window and quality falls.

Each one has a fix. All four fixes are Module 2.



Module 2 is the one that does not get cut.



Six lines to keep.

1. A loop is a state machine you enforce.
2. Verify is what separates a loop from a generator.
3. Write scope is a missing method, not a polite request.
4. Three exits. The forgotten one is stable failure.
5. Memory lives on disk. The window is not it.
6. The human still owns irreversible action.

Break. 15 minutes.

Next: the harness. Two doers, a red gate, ten rubric rows, and a gate that refuses to let you push.

Module 2 is the one that does not get cut.

Primary references for this session.

- Yao et al. ReAct. arXiv:2210.03629
- Ridnik, Kredo, Friedman. AlphaCodium. arXiv:2401.08500
- Liu et al. Lost in the Middle. TACL 2024. arXiv:2307.03172
- `loops/enhancer.py` , `loops/gates.py` , `loops/roles.py` , `loops/criteria.py`
- `labs/lab1_enhancer/ARCHITECTURE.md`